

Contents

Preface	3
1 Python Foundations	9
1.1 Reasons for Python's Popularity	9
1.2 Python Features	9
1.3 Where is Python Used?	10
1.4 Who Uses Python?	10
1.5 Programming Paradigms	10
1.6 Python as an Interpreted Language	11
1.7 Python Resources and IDLE	11
1.8 Python Programming Modes	11
1.8.1 Interactive Mode	11
1.8.2 Script Mode	12
1.8.3 Notebook Mode (Jupyter)	12
1.9 Third-party Packages for Data Science	12
1.10 Sophisticated Tools for Data Science	12
1.11 Python Syntax	13
1.12 Python Semantics	13
1.13 Structure of a Simple Python Program	13
1.14 Python Tokens	13
1.15 Keywords	14
1.16 Identifiers	14
1.17 Literals	14
1.18 Statements and Expressions	14
1.19 Variables and Assignment	15
1.20 Multiple Assignment	15
1.21 Indentation	15
1.22 Operators in Python	15
1.22.1 Arithmetic Operators	16
1.22.2 Comparison Operators	16
1.22.3 Assignment Operators	16
1.22.4 Logical Operators	17
1.22.5 Bitwise and Shift Operators	17
1.22.6 Membership Operators	17
1.22.7 Identity Operators	17
1.22.8 Conditional Operator (Ternary)	18
1.22.9 Assignment Expression Operator (:=)	18
1.22.10 Matrix Multiplication Operator (@)	18
1.22.11 Operator Precedence	18
1.22.12 Common Syntax Forms	18
1.23 Complete Program: Student Result Processing	19
1.24 Errors in Python	19
1.24.1 Syntax Error	19
1.24.2 Runtime Error	19
1.24.3 Semantic Error	19

1.25	Exercises — Part A (Syntax, Semantics, Operators)	20
1.26	Control Structures in Python	21
1.26.1	Sequential Control Structure	21
1.26.2	Selection Control Structure	22
1.26.3	The <code>if</code> Statement	22
1.26.4	The <code>if-else</code> Statement	22
1.26.5	The <code>if-elif-else</code> Statement	22
1.26.6	Nested <code>if</code> Statement	22
1.26.7	The <code>match-case</code> Statement	22
1.26.8	Iteration Control Structure	23
1.26.9	The <code>while</code> Loop	23
1.26.10	The <code>for</code> Loop	23
1.26.11	The <code>range()</code> Function	23
1.26.12	Loop with Accumulator	24
1.26.13	The <code>break</code> Statement	24
1.26.14	The <code>continue</code> Statement	24
1.26.15	The <code>pass</code> Statement	24
1.26.16	Loop <code>else</code> Clause	25
1.26.17	Nested Loops	25
1.27	Functions in Python	26
1.27.1	Defining and Calling a Function	26
1.27.2	Parameters and Arguments	26
1.27.3	Function with Return Value	26
1.27.4	Void Function and <code>None</code>	26
1.27.5	Required, Keyword, Default Arguments	26
1.27.6	Variable-Length Arguments	27
1.27.7	Positional-Only and Keyword-Only Parameters	27
1.27.8	Type Hints and Docstrings	27
1.27.9	Scope of Variables (LEGB Rule)	27
1.27.10	Pure and Impure Functions	27
1.27.11	Composition of Functions	28
1.27.12	Recursive Function	28
1.27.13	Lambda Function	28
1.28	Scripting in Python	28
1.28.1	Interactive Mode vs. Script Mode	28
1.28.2	Recommended Structure of a Python Script	28
1.28.3	Meaning of <code>__name__ == "__main__"</code>	29
1.28.4	Importing Modules in Scripts	29
1.28.5	Command-Line Arguments	29
1.28.6	A Better Script Using <code>argparse</code>	29
1.28.7	Complete Script: Mark Report	29
1.29	Exercises — Part B (Control, Functions, Scripting)	29
1.30	Data Types and Algorithmic Problem-Solving	31
1.30.1	Classification of Python Data Types	31
1.30.2	Finding and Checking Types	31
1.30.3	Numeric Data Types	31
1.30.4	Type Conversion	32
1.31	String Data Type	32
1.31.1	String Indexing	32
1.31.2	String Slicing	32
1.31.3	String Immutability	33
1.31.4	String Operators and Methods	33

1.31.5	Algorithmic Examples with Strings	33
1.32	List Data Type	33
1.32.1	Common List Methods	34
1.32.2	Algorithmic Examples with Lists	34
1.33	Tuple Data Type	34
1.33.1	Returning Multiple Values Using Tuple	34
1.34	Dictionary Data Type	34
1.34.1	Common Dictionary Methods	34
1.34.2	Frequency Counting Using Dictionary	35
1.35	Set Data Type	35
1.36	Range, NoneType, Mutable/Immutable, Aliasing	35
1.36.1	Range Data Type	35
1.36.2	NoneType	35
1.36.3	Mutable and Immutable Data Types	35
1.36.4	Aliasing and Copying	35
1.36.5	Choosing the Correct Data Type	35
1.37	Complete Algorithmic Example: Student Mark Analysis	36
1.38	Mini Case Studies	36
1.38.1	Removing Duplicate Survey Responses	36
1.38.2	Student Record Lookup	36
1.39	Programming Patterns	36
1.40	Exercises — Part C (Data Types)	36
1.41	Basic I/O and File Handling for Data Processing	38
1.41.1	Types of Input and Output in Python	38
1.42	Screen Output Using print()	38
1.42.1	Using sep and end in print()	38
1.42.2	Formatted Output Using f-Strings	38
1.43	Keyboard Input Using input()	38
1.43.1	Input Validation	39
1.44	File Handling in Python	39
1.44.1	Basic File Processing Flow	39
1.44.2	File Opening Modes	39
1.45	Writing Text Data to a File	39
1.46	Reading Text Data from a File	40
1.47	Appending Data to a File	40
1.48	File Paths	40
1.49	Exception Handling in File Operations	40
1.50	Text File Data Processing	41
1.50.1	Counting Lines, Words, and Characters	41
1.50.2	Filtering Lines from a File	41
1.50.3	Writing Processed Output to Another File	41
1.51	CSV File Handling	41
1.51.1	Reading CSV Using csv.reader	41
1.51.2	Skipping Header Row Using next()	42
1.51.3	Processing CSV Data: Total and Average	42
1.51.4	Writing Data to a CSV File	42
1.51.5	Reading CSV as Dictionary: DictReader	42
1.51.6	Writing Dictionary Data: DictWriter	42
1.52	JSON File Handling	43
1.52.1	Writing JSON Data	43
1.52.2	Reading JSON Data	43
1.53	Mini Case Studies	43

1.53.1 Processing Student Marks from CSV	43
1.53.2 Cleaning a Text Dataset	44
1.54 File Pointer: tell() and seek()	44
1.55 Text Mode and Binary Mode	44
1.56 Best Practices in File Handling	44
1.57 Complete Integrated Example	44
1.58 Exercises — Part D (I/O and File Handling)	45

Python Foundations

Python is one of the most widely used programming languages for data science because it is simple to read, easy to write, and powerful for numerical and data-oriented work. Before learning advanced topics such as NumPy, pandas, visualization, machine learning, and time series analysis, every student must understand the foundations of Python programming.

What is Python?

Python is a high-level, interpreted, dynamically typed, general-purpose programming language created by Guido van Rossum at the Centrum Wiskunde & Informatica (CWI), a national research institute in the Netherlands. It was first released in 1991.

Timeline of Python:

1991 -> Python 1.0 released by Guido van Rossum
 2000 -> Python 2.0 released (**list** comprehensions, garbage collection)
 2008 -> Python 3.0 released (major redesign; current standard)
 2020 -> Python 2.x officially retired

Name origin:

Named after the BBC comedy series "Monty Python's Flying Circus" (early 1970s), NOT after the snake.

Note

Python programmers are often called **Pythonistas** or **Pythonists**. The official Python style guide is Python Enhancement Proposal (PEP) 8. Python interpreters are available for Windows, Linux, macOS, and many other operating systems.

1.1 Reasons for Python's Popularity

Python has become the dominant language in data science and AI for a combination of technical and practical reasons.

Free and open-source No license cost; supported by a global community.

Software quality Readable, reusable, maintainable code.

Developer productivity Much less code than Java or C++, with no compile step.

Program portability Runs unchanged on Windows, Linux, and macOS.

Library support Vast standard and third-party library ecosystem.

Component integration Can call C, C++, Java, and R code directly.

Ease of use Type and run; provides an interactive programming experience.

Productivity comparison (approximate lines of code **for** the same task):
 Java -> 15-20 lines
 C++ -> 10-15 lines
 Python -> 3-5 lines

Python's guiding philosophy (Tim Peters, "The Zen of Python"):

"Beautiful is better than ugly."
 "Simple is better than complex."
 "Readability counts."

Run `import this` in Python to see the full list.

Note

In statically typed languages like C or Java, the programmer must compile the source code into machine code before running it. Python skips this step: the interpreter reads and executes source code directly, making the edit-run cycle much faster during development.

1.2 Python Features

Key Python Features

Python's power comes from a combination of dynamic typing, automatic memory management, a rich standard library, and first-class support for multiple programming paradigms.

Feature	Meaning
Dynamic typing	No variable declarations; types are determined at runtime
Automatic memory management	Garbage collector frees unused memory automatically
Classes and modules	Full object-oriented and modular programming support
Exception handling	Structured error detection and recovery
Built-in operations	Joining, slicing, sorting, mapping, filtering on sequences
Standard library	Hundreds of ready-to-use modules (math, os, json, re, etc.)
Third-party libraries	Tens of thousands of packages via pip (PyPI)
Interactive mode	Type-and-run; immediate feedback without saving a file

1.3 Where is Python Used?

Domain	Use Case
System programming	Scripting, automation, file management
GUI applications	Desktop tools using Tkinter, PyQt, wxPython
Internet scripting	Web servers, REST APIs, web scraping
Database programming	SQL and NoSQL database access
Numeric and scientific	NumPy, SciPy, SymPy for mathematics and physics
Data science and ML	pandas, scikit-learn, TensorFlow, PyTorch
Game programming	pygame, Panda3D
Robotics	ROS (Robot Operating System) supports Python
Big data and AI	PySpark, Dask, Hugging Face transformers
Rapid prototyping	Quick proof-of-concept before production implementation

1.4 Who Uses Python?

Python is used by some of the world's largest technology companies and research institutions.

Organisation	Use of Python
Google	Web search system, internal tools
YouTube	Video sharing and recommendation platform
BitTorrent	Peer-to-peer file sharing system
Intel, HP, IBM, Qualcomm	Hardware testing and automation
Pixar, Industrial Light & Magic	Movie animation pipelines
JP Morgan, UBS	Financial market forecasting
NASA, Fermi-Lab	Scientific programming and data analysis
iRobot	Commercial robot vacuum cleaners
NSA	Cryptographic and intelligence analysis

1.5 Programming Paradigms

Programming Paradigm

A programming paradigm is an organizational principle or model that defines the structure and style of building computer programs. It is a fundamental approach to expressing algorithms and organizing code.

Python is a multi-paradigm language: the same problem can be solved using different approaches depending on what is most natural.

Paradigm	Description and Python Support
Procedural	Step-by-step sequence of instructions; the default Python style
Functional	Functions as first-class objects; <code>map()</code> , <code>filter()</code> , <code>lambda</code>
Object-Oriented	Data and behaviour grouped into classes and objects
Event-Driven	Code runs in response to user actions or system events (GUI)

```

1 # The same task in three Python paradigms
2 # Task: given three marks, print Pass or Fail for each student.
3
4 marks = [72, 45, 88]
5
6 # --- Paradigm 1: Procedural (step-by-step instructions) ---
7 print("Procedural:")
8 for mark in marks:
9     if mark >= 50:
10         print(mark, "-> Pass")
11     else:
12         print(mark, "-> Fail")
13
14 # --- Paradigm 2: Functional (define a function, then apply it) ---
15 def check_result(mark):
16     return "Pass" if mark >= 50 else "Fail" # reusable rule
17
18 print("Functional:")
19 for mark in marks:
20     print(mark, "->", check_result(mark)) # apply the function
21
22 # --- Paradigm 3: Object-Oriented (group data and behaviour) ---
23 class Student:
24     def __init__(self, name, mark):
25         self.name = name # data stored inside the object
26         self.mark = mark
27
28     def result(self): # behaviour stored inside the object
29         return "Pass" if self.mark >= 50 else "Fail"
30
31 students = [Student("Anu", 72), Student("Ravi", 45), Student("Meena", 88)]
32
33 print("Object-Oriented:")
34 for s in students:
35     print(s.name, s.mark, "->", s.result())
36
37 Procedural:
38 72 -> Pass
39 45 -> Fail
40 88 -> Pass
41 Functional:
42 72 -> Pass
43 45 -> Fail
44 88 -> Pass
45 Object-Oriented:
46 Anu 72 -> Pass
47 Ravi 45 -> Fail
48 Meena 88 -> Pass

```

All three produce the same result using different paradigms.

Procedural -> write every step directly.

Functional -> wrap the rule **in** a named function **and** **apply** it.

OO -> bundle data **and** behaviour together **in** a **class**.

Python does **not** force one paradigm; use whichever is clearest.

1.6 Python as an Interpreted Language

Interpreted Language

In an interpreted language, the source code is executed directly by an interpreter at runtime, without a separate compilation step. Each statement is translated to machine instructions and executed immediately.

```
Compiled Language (e.g., C, C++, Java):
Source code (.c / .java)
|
v compile (separate step, may take seconds or
minutes)
|
Machine code / bytecode
|
v run
|
Output

Interpreted Language (Python):
Source code (.py)
|
v interpreter reads and executes line by line (no
separate step)
|
Output

Python also compiles to bytecode (.pyc) internally,
but this is transparent to the programmer.
```

Property	Compiled	Interpreted
Speed	Faster execution	Slower (overhead at runtime)
Development cycle	Longer (compile then run)	Shorter (run immediately)
Portability	Depends on platform	Source runs anywhere with interpreter
Error detection	At compile time	At runtime
Examples	C, C++, Java	Python, Ruby, JavaScript

Note

Python is Also Dynamic Python is simultaneously high-level, interpreted, dynamically typed, and object-oriented. "Dynamically typed" means variables do not have fixed types; the same variable name can hold an integer at one point and a string at another.

1.7 Python Resources and IDLE

Resource	Location
Python download	www.python.org
Documentation	docs.python.org
Package index (PyPI)	pypi.org
Tutorial	docs.python.org/3/tutorial

Python program development can be done in three ways:

1. using the built-in IDLE (Integrated Development and Learning Environment);
2. using a third-party IDE such as PyCharm, VS Code, or Spyder;
3. using online Python shells such as Google Colab, Replit, or Jupyter.

IDLE

IDLE is Python's built-in development environment. It provides syntax highlighting, a code editor, an interactive shell, and a basic debugger. It is installed automatically with Python.

IDLE features:

1. syntax highlighting (keywords, functions, strings in different colours);
2. context-sensitive help (press F1 on any name);
3. interactive shell for instant code testing;
4. basic debugger for stepping through programs.

1.8 Python Programming Modes

Python code can be written and executed in three distinct modes.

Mode	Description
Interactive mode	Type one statement at a time in the Python shell; result shown immediately
Script mode	Write a complete program in a .py file; run the entire file
Notebook mode	Write code and text in a Jupyter notebook (.ipynb); run cell by cell

1.8.1 Interactive Mode

Interactive Mode

In interactive mode, the Python interpreter reads one expression or statement at a time, evaluates it immediately, and displays the result. The prompt `>>>` indicates the interpreter is waiting for input.

```
>>> 2 + 3
5
>>> "Hello" + " " + "Python"
'Hello Python'
>>> x = 10
>>> x * x
100
>>> type(x)
<class 'int'>
```

Interactive mode **is** ideal **for**:

- testing a small idea quickly
- checking the output of a single expression
- exploring library functions

1.8.2 Script Mode

Script Mode

In script mode, all statements are written in a plain text file with the `.py` extension. The entire file is passed to the interpreter for execution. This is the standard mode for programs meant to be reused or shared.

```
1 # hello.py -- a simple Python script
2 name = input("Enter your name: ") # prompt user
3 print("Hello,", name) # display greeting
```

Run **from** terminal:
python hello.py

Output:

```
Enter your name: Anu
Hello, Anu
```

Script mode **is** ideal **for**:

- programs run repeatedly
- programs shared **with** others
- programs larger than a few lines

1.8.3 Notebook Mode (Jupyter)

Notebook Mode

Jupyter notebooks (`.ipynb`) combine executable code cells with Markdown text cells, mathematical equations, and inline plots. This mode is the standard in data science and research.

Mode	File	Run Style	Best For
Interactive	none	one line at a time	quick testing, exploration
Script	<code>.py</code>	whole file at once	reusable programs, automation
Notebook	<code>.ipynb</code>	cell by cell	data science, teaching, reports

1.9 Third-party Packages for Data Science

Python's power in data science comes from its ecosystem of third-party packages, installed via the package manager `pip`.

Python's data science ecosystem is built on a set of well-established third-party packages. **NumPy** provides fast N-dimensional arrays and mathematical functions for numerical computing. Built on top of NumPy, **pandas** introduces the `Series` and `DataFrame` structures for tabular data analysis. For visualization, **Matplotlib** offers flexible 2D and 3D plotting with fine-grained control, while **Seaborn** provides statistical plots with cleaner default styles. **scikit-learn** covers the core machine learning tasks: classification, regression, clustering, and preprocessing. For

deep learning, **TensorFlow** (developed by Google) and **PyTorch** (developed by Meta) are the two dominant frameworks for building and training neural networks. **SciPy** extends NumPy with tools for integration, optimization, and signal processing. **SymPy** handles symbolic mathematics such as algebra and calculus entirely within Python. Finally, **Statsmodels** supports statistical modelling and hypothesis testing.

> Syntax: Installing Packages

```
pip install numpy # install NumPy
pip install pandas # install pandas
pip install matplotlib # install Matplotlib
pip install scikit-learn # install scikit-learn
pip install jupyter # install Jupyter
```

1.10 Sophisticated Tools for Data Science

Tool	Purpose
Jupyter Notebook	Interactive notebook combining code, text, equations, and plots
JupyterLab	Modern IDE-like interface built on Jupyter
Anaconda	Python distribution bundling 300+ data science packages and Conda package manager
Spyder	Scientific Python IDE with variable explorer and debugger
VS Code	General-purpose editor with excellent Python and Jupyter support
Google Colab	Free cloud-based Jupyter notebook with GPU access

Note

Anaconda is the most popular Python distribution for data science. It installs Python, Jupyter, NumPy, pandas, scikit-learn, Matplotlib, Seaborn, and over 300 other packages in one step, avoiding dependency conflicts.

Learning Objectives

After completing this unit, the learner will be able to:

1. understand Python syntax and semantics;
2. use variables, constants, operators, and expressions;
3. apply elementary mathematics in Python programs;
4. write conditional and loop-based programs;
5. use strings, lists, tuples, sets, and dictionaries for data processing;
6. define and call Python functions;
7. write small Python scripts;
8. read input from the keyboard and display formatted output;

9. perform basic file handling for text, CSV, and JSON data;
10. debug beginner-level Python programs.

1.11 Python Syntax

Syntax means the grammar rules of a programming language. In English, grammar tells us how to arrange words in a meaningful sentence. Similarly, Python syntax tells us how to arrange symbols, words and statements so that the Python interpreter can understand them.

Definition

Python syntax is the set of rules that decides whether a Python program is written in a valid form.

➤ Syntax: Basic Output

```
print(value)
```

⇒ Program: Correct and Incorrect Syntax

```
1 # Correct syntax -- closing parenthesis present
2 print("Welcome to Python") # displays the message
3
4 # Incorrect syntax -- missing closing parenthesis
5 # print("Welcome to Python" # SyntaxError!
```

Output

```
Welcome to Python
```

1.12 Python Semantics

Semantics means the meaning or logic of a program. A program may be syntactically correct but semantically wrong.

Definition

Python semantics refers to what a Python statement actually *means* during execution.

⇒ Program: Semantic Error: Wrong Formula

```
1 # Program to calculate the area of a rectangle
2
3 length = 10
4 breadth = 5
5
6 # WRONG semantics: area needs multiplication, not
  addition
7 area_wrong = length + breadth # gives 15 (
  incorrect)
8
9 # CORRECT semantics
10 area_correct = length * breadth # gives 50 (correct
   )
11
12 print("Wrong area =", area_wrong)
13 print("Correct area =", area_correct)
```

Output

```
Wrong area = 15
Correct area = 50
```

🔗 Syntax vs. Semantics

Syntax	Is the program grammatically correct?
Semantics	Does it solve the right problem?

1.13 Structure of a Simple Python Program

A simple Python program usually follows the Input-Process-Output model.

IPO Model

Input → Processing → Output

Start | Input | Processing | Output | Stop

⇒ Program: Addition of Two Numbers

```
1 # Read two numbers and add them
2
3 # INPUT: read from keyboard, convert to int
4 first_number = int(input("Enter first number: "))
5 second_number = int(input("Enter second number: "))
6
7 # PROCESS: compute the sum
8 sum_value = first_number + second_number
9
10 # OUTPUT: display the result
11 print("Sum =", sum_value)
```

Output

```
Enter first number: 12
Enter second number: 18
Sum = 30
```

1.14 Python Tokens

A **token** is the smallest meaningful unit of a program. A Python program is formed by combining many tokens.

Python Program

```
|
v
Tokens
+-- Keywords
+-- Identifiers
+-- Literals
+-- Operators
+-- Delimiters
```

In the statement `total = mark1 + mark2:`

Token	Type
total	Identifier
=	Assignment operator
mark1	Identifier
+	Arithmetic operator
mark2	Identifier

1.15 Keywords

Definition

A **keyword** is a reserved word that has a special, predefined meaning in Python syntax. Keywords cannot be used as variable names.

False	None	True	and	as
assert	async	await	break	class
continue	def	del	elif	else
except	finally	for	from	global
if	import	in	is	lambda
nonlocal	not	or	pass	raise
return	try	while	with	yield

Program: Correct Use of Identifiers vs. Keywords

```
1 # WRONG: 'class' is a keyword -> SyntaxError
2 # class = 12
3
4 # CORRECT: add underscore to avoid conflict
5 class_number = 12
6 print(class_number)    # 12
```

Using Keywords as Variable Names

Do not use class, if, for, while, return, or import as variable names. They have fixed meanings in Python syntax.

1.16 Identifiers

An identifier is a name given to a variable, function, class, module, or object.

Syntax: Identifier Rules

- May contain letters, digits, underscore (_)
- Must NOT start **with** a digit
- Must NOT contain spaces **or** special symbols
- Must NOT be a keyword
- Case-sensitive: name != Name != NAME

Valid	Invalid (reason)
name	1mark (starts with digit)
student_name	student name (has space)
_result	total-marks (hyphen)
mark1	class (keyword)
area2	mark\$ (symbol)

Program: Case Sensitivity

```
1 # Demonstration of case sensitivity
2 name = "Ravi"    # lowercase 'name'
3 Name = "Kumar"   # different identifier 'Name'
4
5 print(name)      # Ravi
6 print(Name)     # Kumar -- different variable!
```

Output

```
Ravi
Kumar
```

PEP 8 Naming Conventions

Variables / functions	snake_case
Constants	ALL_CAPS
Classes	CamelCase

Element	Convention	Example
Variables / Functions	snake_case	student_name, calculate_grade()
Constants	ALL_CAPS	MAX_SCORE, DATABASE_URL
Classes	CamelCase	StudentRecord, DatabaseConnection

Convention	Purpose
snake_case	Improves readability for variables/functions
ALL_CAPS	Makes constants stand out as unchangeable
CamelCase	Clearly identifies class definitions

1.17 Literals

A **literal** is a fixed value written directly in a program.

Type	Example
Integer	10, -25
Float	3.14, -7.5
Complex	3+4j
String	"Python"
Boolean	True, False
None	None
List	[10, 20, 30]
Tuple	(10, 20, 30)
Set	{10, 20, 30}
Dictionary	{"name": "Ravi"}

Program: Different Types of Literals

```
1 age = 17          # integer literal
2 height = 5.8      # floating-point literal
3 name = "Anu"      # string literal
4 is_student = True  # boolean literal
5 result = None     # special None literal
6
7 # type() reveals the class of each literal
8 print(type(age))   # <class 'int'>
9 print(type(height)) # <class 'float'>
10 print(type(name))  # <class 'str'>
11 print(type(is_student)) # <class 'bool'>
12 print(type(result)) # <class 'NoneType'>
13
14 # Print the values themselves
15 print(age, height, name, is_student, result)
```

Output

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
<class 'NoneType'>
17 5.8 Anu True None
```

1.18 Statements and Expressions**Statement**

A **statement** performs an action. Example: `x = 10`

Expression

An **expression** produces a value. Example: `10 + 20`

Program: Statement and Expression

```
1 # Right side is an EXPRESSION -- evaluated first
2 # The result (30) is then stored by the STATEMENT
3 x = 10 + 20 # expression 10+20 = 30; stored in x
4
5 print(x)    # 30
```

Output

```
30
```

Output

```
10
20
30
a = 20
b = 10
```

1.19 Variables and Assignment

A variable is a name used to refer to a value stored in memory.

Syntax: Assignment Statement

```
variable_name = value
```

Important

In mathematics, = means equality. In Python, = means **assignment**: store the value on the right into the variable on the left.

Program: Variable Assignment and Update

```
1 # Store a value in a variable
2 age = 17
3 print(age)    # 17
4
5 # Updating: read old value, compute, store new value
6 x = 5
7 x = x + 2     # old x=5, add 2, store 7 in x
8 print(x)     # 7
```

Output

```
17
7
```

Syntax: Indented Block

```
if condition:
    statement inside block
    statement inside block
statement outside block
```

Program: Indentation in Action

```
1 # Checking pass or fail
2 mark = 80
3
4 if mark >= 35:
5     print("Pass")           # inside if-block (
6     print("Congratulations") # also inside if-block
7
8 # This line is outside the if-block (not indented)
9 print("Check complete")
```

Output

```
Pass
Congratulations
Check complete
```

1.20 Multiple Assignment

Syntax: Multiple Assignment

```
var1, var2, var3 = value1, value2, value3
a = b = c = value    # chain: all same object
```

Program: Multiple Assignment and Swap

```
1 # Assign different values simultaneously
2 a, b, c = 10, 20, 30
3 print(a)    # 10
4 print(b)    # 20
5 print(c)    # 30
6
7 # Elegant swap -- Python evaluates RHS first
8 a = 10
9 b = 20
10 a, b = b, a    # swap without a temporary
    variable
11 print("a =", a)    # a = 20
12 print("b =", b)    # b = 10
```

X Indentation Error

```
1 if mark >= 35:
2     print("Pass")    # IndentationError!
3
4 # Correct form:
5 if mark >= 35:
6     print("Pass")    # 4 spaces indent
```

1.22 Operators in Python

Operators are symbols or keywords that perform operations on operands. In `a + b`: `a` and `b` are operands; `+` is the operator.

Python Operators

++ Arithmetic operators	++ Membership operators
++ Comparison operators	++ Identity operators
++ Assignment operators	++ Conditional operator
++ Logical operators	++ Assignment expression (:=)
++ Bitwise operators	++ Matrix multiplication (@)

1.22.1 Arithmetic Operators

Op	Meaning	Example ($a = 10, b = 3$)
+	Addition	13
-	Subtraction	7
*	Multiplication	30
/	True division	3.333...
//	Floor division	3
%	Modulo	1
**	Power	1000
@	Matrix mult.	A@B

Program: Arithmetic Operators

```
1 # Demonstration of arithmetic operators
2 a = 10
3 b = 3
4
5 print("Addition =", a + b) # 13
6 print("Subtraction =", a - b) # 7
7 print("Multiplication =", a * b) # 30
8 print("Division =", a / b) # 3.333... always
9 # float
10 print("Floor Division =", a // b) # 3 integer
11 # quotient
12 print("Remainder =", a % b) # 1 remainder
13 print("Power =", a ** b) # 1000
```

Output

```
Addition = 13
Subtraction = 7
Multiplication = 30
Division = 3.3333333333333335
Floor Division = 3
Remainder = 1
Power = 1000
```

🔗 Difference Between /, // and %

For 17 divided by 5:

```
17 / 5 = 3.4 (true division -- always float)
17 // 5 = 3 (floor quotient -- integer)
17 % 5 = 2 (remainder)
Identity: a == (a//b)*b + (a%b)
```

🔗 Math Bridge: Remainder and Divisibility

If a and b are integers ($b \neq 0$), the division algorithm gives:

$$a = b \cdot q + r, \quad 0 \leq r < |b|$$

where $q = a // b$ (quotient) and $r = a \% b$ (remainder).

Divisibility: b divides a iff $a \% b == 0$.

Example: $17 = 5 \times 3 + 2$, so $17 // 5 = 3$ and $17 \% 5 = 2$.

🔗 Math Bridge: Even and Odd

n is **even** if $n \bmod 2 = 0$; **odd** if $n \bmod 2 = 1$.

Program: Even or Odd

```
1 number = int(input("Enter an integer: "))
2
3 # n % 2 gives remainder when divided by 2
4 if number % 2 == 0: # remainder 0 => even
5     print("Even number")
6 else: # remainder 1 => odd
7     print("Odd number")
```

Output

```
Enter an integer: 27
Odd number
```

1.22.2 Comparison Operators

Comparison operators compare two values and return True or False.

Op	Meaning
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

Program: Comparison Operators

```
1 a = 100
2 b = 35
3
4 print(a == b) # False equal?
5 print(a != b) # True not equal?
6 print(a > b) # True greater?
7 print(a < b) # False less?
8 print(a >= b) # True
9 print(a <= b) # False
10
11 # Chained comparison -- very Pythonic
12 mark = 85
13 print(0 <= mark <= 100) # True: valid mark range
```

Output

```
False
True
True
False
True
False
True
```

➤ Syntax: Chained Comparison

```
lower_limit <= value <= upper_limit
```

1.22.3 Assignment Operators

Op	Equivalent
=	simple assignment
+=	$x = x + \text{value}$
-=	$x = x - \text{value}$
*=	$x = x * \text{value}$
/=	$x = x / \text{value}$
//=	$x = x // \text{value}$
%=	$x = x \% \text{value}$
**=	$x = x ** \text{value}$
&=	$x = x \& \text{value}$
=	$x = x \text{value}$
^=	$x = x ^ \text{value}$
<<=	$x = x \ll \text{value}$
>>=	$x = x \gg \text{value}$

Program: Assignment Operators

```
1 x = 10
2
3 x += 5 # x = 10 + 5 = 15
```

```

4 print(x)      # 15
5
6 x *= 2        # x = 15 * 2 = 30
7 print(x)      # 30
8
9 x -= 4        # x = 30 - 4 = 26
10 print(x)     # 26

```

Output

```

15
30
26

```

Output

```

1
7
6
-6
10
2

```

1.22.4 Logical Operators

Op	Meaning
and	True when <i>both</i> conditions are true
or	True when <i>at least one</i> is true
not	Reverses the truth value

Program: Logical Operators

```

1 age = 18
2 mark = 85
3
4 # and: both must be true
5 print(age >= 18 and mark >= 80) # True
6
7 # or: at least one must be true
8 print(age >= 18 or mark >= 90) # True
9
10 # not: reverses the result
11 print(not age >= 18) # False

```

Output

```

True
True
False

```

Short-Circuit Evaluation

In **and**, if the first condition is false, Python does not check the second. In **or**, if the first condition is true, Python does not check the second. This improves performance and prevents errors.

1.22.5 Bitwise and Shift Operators

Bitwise operators work on the binary (bit-level) representation of integers.

Op	Meaning
&	Bitwise AND: 1 only when both bits are 1
	Bitwise OR: 1 when at least one bit is 1
^	Bitwise XOR: 1 when bits differ
~	Bitwise NOT: flips all bits ($\sim a = -(a + 1)$)
<<	Left shift: multiply by 2^n ($x << n = x * 2^n$)
>>	Right shift: floor-divide by 2^n ($x >> n = x // 2^n$)

Program: Bitwise Operators

```

1 # Demonstration of bitwise operators
2 a = 5 # binary: 0101
3 b = 3 # binary: 0011
4
5 print(a & b) # AND -> 0001 = 1
6 print(a | b) # OR -> 0111 = 7
7 print(a ^ b) # XOR -> 0110 = 6
8 print(~a) # NOT -> -(5+1) = -6
9 print(a << 1) # left shift by 1 -> 1010 = 10 (*2)
10 print(a >> 1) # right shift by 1 -> 0010 = 2 (//2)

```

Binary illustration:

```

5 -> 0101
3 -> 0011

5 & 3:
0101
0011
----
0001 -> 1

5 | 3:
0101
0011
----
0111 -> 7

```

1.22.6 Membership Operators

Membership operators test whether a value is present in a sequence or collection.

Op	Meaning
in	True if value is present
not in	True if value is not present

Program: Membership Operators

```

1 numbers = [10, 20, 30, 40]
2
3 print(20 in numbers) # True -- 20 is in the list
4 print(50 in numbers) # False -- 50 is not in the list
5 print(50 not in numbers) # True
6
7 # For dictionaries, 'in' checks KEYS, not values
8 student = {"name": "Ravi", "age": 17}
9 print("name" in student) # True -- "name" is a key
10 print("Ravi" in student) # False -- "Ravi" is a value, not a key

```

Output

```

True
False
True
True
False

```

Note

For dictionaries, the **in** operator checks **keys**, not values.

1.22.7 Identity Operators

Identity operators check whether two variables refer to the *same object* in memory.

Op	Meaning
is	True if both refer to the same object
is not	True if they refer to different objects

Program: Identity Operators

```
1 a = [1, 2, 3]
2 b = a           # b points to THE SAME list as a
3 c = [1, 2, 3]   # c is a DIFFERENT list object
4
5 print(a == b)    # True (equal values)
6 print(a is b)    # True (same object in memory!)
7
8 print(a == c)    # True (equal values)
9 print(a is c)    # False (different objects)
```

Output

```
True
True
True
False
```

Preferred Use

Use is mainly when comparing with None:

```
1 if result is None:
2     print("No value available")
```

1.22.8 Conditional Operator (Ternary)

Syntax: Conditional Expression

```
value_if_true if condition else value_if_false
```

Program: Conditional Operator

```
1 a = 10
2 b = 20
3
4 # Select the larger value in one line
5 maximum = a if a > b else b # condition is False ->
6                               b selected
7 print("Maximum =", maximum) # 20
8
9 # One-line pass/fail
10 mark = 45
11 status = "Pass" if mark >= 35 else "Fail"
12 print(status) # Pass
```

Output

```
Maximum = 20
Pass
```

1.22.9 Assignment Expression Operator (:=)

The assignment expression operator `:=` is called the **walrus operator**. It assigns a value inside an expression.

Syntax: Assignment Expression

```
variable := expression
```

Program: Walrus Operator

```
1 word = "Python"
2
3 # len(word) is computed, stored in 'length',
4 # AND used in the condition -- all in one step
5 if (length := len(word)) > 5:
6     print("Long word")
7     print("Length =", length) # 6
```

Output

```
Long word
Length = 6
```

Use Carefully

The walrus operator is useful in advanced situations, but it can reduce readability. For beginners, ordinary assignment is usually clearer.

1.22.10 Matrix Multiplication Operator (@)

The `@` operator performs matrix multiplication (mainly used with NumPy).

Syntax: Matrix Multiplication

```
A @ B
```

Program: Matrix Multiplication with NumPy

```
1 import numpy as np
2
3 # Create two 2x2 matrices
4 A = np.array([[1, 2], [3, 4]])
5 B = np.array([[5, 6], [7, 8]])
6
7 C = A @ B # row-by-column multiplication
8
9 print(C)
10 # Row 0, Col 0: 1*5 + 2*7 = 19
11 # Row 0, Col 1: 1*6 + 2*8 = 22
12 # Row 1, Col 0: 3*5 + 4*7 = 43
13 # Row 1, Col 1: 3*6 + 4*8 = 50
```

Output

```
[[19 22]
 [43 50]]
```

1.22.11 Operator Precedence

Priority (high → low)	Operators
1 (highest)	(), indexing, function call
2	**
3	+x, -x, ~x (unary)
4	*, @, /, //, %
5	+, -
6	<<, >>
7	&
8	^
9	
10	comparisons, is, in
11	not
12	and
13 (lowest)	or

Program: Operator Precedence

```
1 # * has higher priority than +
2 result = 10 + 5 * 2
3 print(result) # 20 (not 30)
4
5 # Parentheses override precedence
6 result = (10 + 5) * 2
7 print(result) # 30
8
9 # ** is right-associative
10 print(2 ** 3 ** 2) # 2**(3**2) = 2**9 = 512
```

Output

```
20
30
512
```

1.22.12 Common Syntax Forms

Output Statement

> Syntax: print()

```
print(value1, value2, sep=' ', end='\n')
```

⇒ Program: print() with sep and end

```
1 # sep changes the separator between values
2 print("Python", "Programming", sep="-")
3
4 # end changes what appears after the last value
5 print("Hello", end=" ") # cursor stays on same line
6 print("Students") # continues on same line
```

Output

```
Python-Programming
Hello Students
```

Input Statement

> Syntax: input()

```
variable = input("prompt message")
```

⇒ Program: Reading Name and Age

```
1 # input() ALWAYS returns a string
2 name = input("Enter your name: ") # string
3 age = int(input("Enter your age: ")) # convert to int
4
5 print("Name:", name)
6 print("Age:", age)
```

📌 Note

The `input()` function always returns a `str`. Use `int()` or `float()` when numerical input is required.

1.23 Complete Program: Student Result Processing

⇒ Program: Student Result Processing

```
1 # Student result processing program
2
3 student_name = input("Enter student name: ")
4
5 # Read three subject marks and convert to int
6 maths = int(input("Enter Maths mark: "))
7 physics = int(input("Enter Physics mark: "))
8 computer = int(input("Enter Computer Science mark: "))
9
10 # Calculate total and average
11 total = maths + physics + computer
12 average = total / 3
13
14 # Check pass/fail: all subjects must be >= 35
15 if maths >= 35 and physics >= 35 and computer >= 35:
16     result = "Pass"
17 else:
18     result = "Fail" # any mark below 35 => Fail
19
20 # Assign grade based on average
21 if average >= 90:
22     grade = "A+"
23 elif average >= 75:
24     grade = "A"
25 elif average >= 60:
26     grade = "B"
27 elif average >= 50:
28     grade = "C"
```

```
29 else:
30     grade = "D"
31
32 # Display the complete result
33 print("Student Name:", student_name)
34 print("Total:", total)
35 print("Average:", average)
36 print("Result:", result)
37 print("Grade:", grade)
```

Output

```
Enter student name: Kavya
Enter Maths mark: 95
Enter Physics mark: 86
Enter Computer Science mark: 91
Student Name: Kavya
Total: 272
Average: 90.66666666666667
Result: Pass
Grade: A+
```

1.24 Errors in Python

1.24.1 Syntax Error

A syntax error occurs when Python grammar is violated. Python detects it *before* execution.

⇒ Program: Syntax Error Example

```
1 # Wrong: missing colon after if condition
2 # if 10 > 5
3 #     print("Yes") # SyntaxError!
4
5 # Correct:
6 if 10 > 5:
7     print("Yes")
```

1.24.2 Runtime Error

A runtime error occurs *while* the program is running.

⇒ Program: Runtime Error Example

```
1 a = 10
2 b = 0
3 # print(a / b) # ZeroDivisionError at runtime!
4
5 # Safe version with check
6 if b != 0:
7     print(a / b)
8 else:
9     print("Cannot divide by zero")
```

1.24.3 Semantic Error

A semantic error occurs when the program runs but gives the *wrong* answer.

⇒ Program: Semantic Error Example

```
1 length = 10
2 breadth = 5
3
4 # WRONG: area is multiplication, not addition
5 # perimeter = length + breadth # 15 (wrong!)
6
7 # CORRECT perimeter formula: 2*(l+b)
8 perimeter = 2 * (length + breadth)
9 print(perimeter) # 30
```

Output

```
30
```


1.25 Exercises — Part A (Syntax, Semantics, Operators)

Part A: Short Answer

Q1. Define syntax. Give one example of a syntax error.

Answer / Program

Syntax = grammar rules of Python. A syntax error means the code violates these rules and Python cannot parse it. Example: `if mark >= 35` (missing colon) causes `SyntaxError`.

Q2. Define semantics. Give one example of a semantic error.

Answer / Program

Semantics = the meaning/logic of a program. A semantic error: the program runs but gives wrong output. Example: using `area = length + breadth` instead of `area = length * breadth`.

Q3. What is a token? Name the five token categories.

Answer / Program

A token is the smallest meaningful unit of a Python program. Categories: Keywords, Identifiers, Literals, Operators, Delimiters.

Q4. What is an identifier? Write the rules.

Answer / Program

An identifier is a programmer-chosen name. Rules: must start with letter or `_`, may contain letters/digits/`_`, cannot be a keyword, case-sensitive.

Q5. Write five Python keywords.

Answer / Program

`if, for, while, def, return`

Q6. What is the difference between `=` and `==`?

Answer / Program

`=` is assignment (stores value). `==` is comparison (tests equality, returns True/False).

Q7. What is the use of `//`? What is the use of `%`?

Answer / Program

`//` gives the floor quotient (integer part). `%` gives the remainder after division. For $17 \div 5$: $17 // 5 = 3$, $17 \% 5 = 2$.

Q8. What is the difference between `is` and `==`?

Answer / Program

`==` checks value equality. `is` checks object identity (same memory location). Use `is` only with `None`.

Q9. What is indentation and why is it important?

Answer / Program

Indentation is leading whitespace (4 spaces) that defines code blocks. Python uses it instead of braces. Wrong indentation causes `IndentationError`.

Q10. What is short-circuit evaluation?

Answer / Program

In `and`: if first operand is `False`, second is not evaluated. In `or`: if first is `True`, second is not evaluated. Saves time and prevents errors.

Part B: Find the Output

♦ Find the Output.

```
x = 10; y = 3
print(x // y)
print(x % y)
```

Answer / Program

3 (floor quotient: $10 = 3 \times 3 + 1$) 1 (remainder)

♦ Find the Output.

```
a = 5; b = 2
print(a ** b)
print(a << 1)
```

Answer / Program

25 (5^2) 10 (left-shift: $5 \times 2 = 10$)

♦ Find the Output.

```
x = [1, 2, 3]; y = x; z = [1, 2, 3]
print(x == z)
print(x is z)
print(x is y)
```

Answer / Program

True (equal values) False (different objects) True (same object: `y=x`)

Part C: Programming Exercises

Q11. Write a Python program to calculate the area and perimeter of a rectangle.

Answer / Program

```
1 l = float(input("Length: "))
2 b = float(input("Breadth: "))
3 print(f"Area = {l*b:.2f}")
4 print(f"Perimeter = {2*(l+b):.2f}")
```

Q12. Write a Python program to calculate simple interest.

Answer / Program

```
1 p = float(input("Principal: "))
2 r = float(input("Rate (%): "))
3 t = float(input("Time (yr): "))
4 si = (p * r * t) / 100
5 print(f"SI = Rs.{si:.2f}")
6 print(f"Amount = Rs.{p+si:.2f}")
```

Q13. Check whether a mark is between 0 and 100.

Answer / Program

```
1 mark = int(input("Enter mark: "))
2 if 0 <= mark <= 100:    # chained comparison
```

```

3     print("Valid mark")
4 else:
5     print("Invalid mark")

```

Q14. Find the greatest of two numbers using the conditional operator.

Answer / Program

```

1 a = int(input("Enter a: "))
2 b = int(input("Enter b: "))
3 maximum = a if a > b else b
4 print("Maximum =", maximum)

```

Q15. Demonstrate all arithmetic operators.

Answer / Program

```

1 a, b = 10, 3
2 print(f"+ {a+b} - {a-b} * {a*b} "
3       f"/ {a/b:.2f} // {a//b} % {a%b} ** {a**b}")

```

Q16. Demonstrate assignment operators.

Answer / Program

```

1 x = 10
2 x += 5; print(x)    # 15
3 x *= 2; print(x)    # 30
4 x -= 4; print(x)    # 26
5 x /= 3; print(x)    # 8
6 x **= 2; print(x)   # 64

```

Q17. Demonstrate bitwise operators.

Answer / Program

```

1 a, b = 5, 3    # 5=0101, 3=0011
2 print(f"AND={a&b} OR={a|b} XOR={a^b} "
3       f"NOT=~a LS={a<<1} RS={a>>1}")
4 # AND=1 OR=7 XOR=6 NOT=-6 LS=10 RS=2

```

Q18. Check whether a word contains a particular character.

Answer / Program

```

1 word = input("Enter a word: ")
2 char = input("Enter character: ")
3 if char in word:    # membership test
4     print(f"'{char}' found in '{word}'")
5 else:
6     print(f"'{char}' NOT found in '{word}'")

```

Q19. Compare identity and equality using two lists.

Answer / Program

```

1 a = [1,2,3]; b = a; c = [1,2,3]
2 print("a==b:", a==b)    # True
3 print("a is b:", a is b) # True (same object)
4 print("a==c:", a==c)    # True
5 print("a is c:", a is c) # False (different)

```

Q20. Read three subject marks and print total, average, result and grade.

Answer / Program

```

1 m=[int(input(f"Mark {i+1}: ")) for i in range(3)]
2 t=sum(m); a=t/3
3 r="Pass" if all(x>=35 for x in m) else "Fail"
4 g="O" if a>=90 else "A+" if a>=80 else "A" \
5   if a>=70 else "B+" if a>=60 else "B" if a>=50 else "F"
6 print(f"Total={t} Avg={a:.2f} Result={r} Grade={g}")

```

1.26 Control Structures in Python

A program is usually not just a straight list of statements. In real problem solving, we must make decisions, repeat statements, skip unnecessary steps, and stop a loop when a condition is satisfied. These ideas are handled by control structures.

Meaning of Control Structure

A **control structure** is a programming construct that controls the order in which statements are executed in a program.

Control Structures

```

+-- Sequential Control
|   +-- Statements execute one after another
|
+-- Selection Control
|   +-- if
|   +-- if-else
|   +-- if-elif-else
|   +-- nested if
|   +-- match-case
|
+-- Iteration Control
|   +-- while loop
|   +-- for loop
|   +-- nested loops
|
+-- Transfer Control
|   +-- break
|   +-- continue
|   +-- pass
|   +-- return

```

Simple Understanding

A control structure answers one of:

1. Should this statement be executed?
2. How many times should this be repeated?
3. Should the loop continue or stop?
4. Should the function return a value now?

1.26.1 Sequential Control Structure

Sequential control is the simplest form: statements execute top to bottom, one after another.

Syntax: Sequential Execution

```

statement_1
statement_2
statement_3
...

```

Program: Sequential Execution

```

1 name = "Kavya"    # 1. store name
2 mark1 = 80         # 2. store first mark
3 mark2 = 90         # 3. store second mark
4 total = mark1 + mark2 # 4. add marks
5 print("Name:", name) # 5. print name
6 print("Total:", total) # 6. print total = 170

```

Output

```

Name: Kavya
Total: 170

```

1.26.2 Selection Control Structure

Meaning

Selection control allows Python to execute a block of statements only when a condition is satisfied.

1.26.3 The if Statement

► Syntax: if Statement

```
if condition:
    statement_1
    statement_2
```

► Program: Simple if

```
1 mark = 75
2
3 # Colon is mandatory; body must be indented
4 if mark >= 35:
5     print("Pass")           # executes: 75 >= 35 is
                             # True
```

Output

Pass

✗ Colon and Indentation

The if line must end with a colon. The statements inside the if block must be indented by 4 spaces.

1.26.4 The if-else Statement

► Syntax: if-else Statement

```
if condition:
    statements_if_true
else:
    statements_if_false
```

► Program: Pass or Fail

```
1 mark = 28
2
3 if mark >= 35:
4     print("Pass")           # skipped: 28 >= 35 is False
5 else:
6     print("Fail")           # executes: else block selected
```

Output

Fail

1.26.5 The if-elif-else Statement

► Syntax: if-elif-else Statement

```
if condition_1:
    statements_1
elif condition_2:
    statements_2
elif condition_3:
    statements_3
else:
    default_statements
```

► Program: Grade Assignment

```
1 average = 82
2
3 if average >= 90:
4     grade = "A+"           # not selected: 82 < 90
```

```
5 elif average >= 75:
6     grade = "A"           # SELECTED: 82 >= 75 is True
7 elif average >= 60:
8     grade = "B"           # skipped after True branch
                             # found
9 elif average >= 50:
10    grade = "C"
11 else:
12    grade = "D"
13
14 print("Grade:", grade)    # A
```

Output

Grade: A

Note

In if-elif-else, Python executes **only the first block whose condition is true**. All remaining branches are skipped.

1.26.6 Nested if Statement

A nested if is an if statement inside another if.

► Syntax: Nested if

```
if outer_condition:
    if inner_condition:
        statements
    else:
        statements
else:
    statements
```

► Program: Scholarship Eligibility

```
1 mark = 92
2 income = 150000
3
4 if mark >= 90:               # outer condition
5     True
6     if income <= 200000:     # inner condition
7         True
8         print("Eligible for scholarship")
9     else:
10        print("High mark, but income too high")
11 else:
12    print("Not eligible based on mark")
```

Output

Eligible for scholarship

1.26.7 The match-case Statement

The match-case statement (Python 3.10+) is used for pattern matching.

► Syntax: match-case

```
match variable:
    case pattern_1:
        statements
    case pattern_2:
        statements
    case _:
        default_statements
```

► Program: Menu Using match-case

```
1 choice = 2
2
3 match choice:
4     case 1:
5         print("Add record")
6     case 2:
```

```

7     print("Search record") # selected: choice ==
8     case 3:
9         print("Delete record")
10    case _: # _ = default case
11        print("Invalid choice")

```

Output

Search record

Note

Use match-case when one variable must be compared with many fixed patterns. For simple two-way decisions, if-else is usually clearer.

1.26.8 Iteration Control Structure

Meaning

An **iteration structure** repeatedly executes a block of code as long as a condition is satisfied or as long as there are items to process. Python provides `while` and `for` loops.

1.26.9 The while Loop

The `while` loop is condition-controlled. It repeats as long as the condition is true.

> Syntax: while Loop

```

while condition:
    statements

```

Program: Printing Numbers 1 to 5

```

1 number = 1 # initialise loop
2     variable
3 while number <= 5: # check condition
4     print(number) # execute body
5     number += 1 # UPDATE: must not
6     forget!
7 # When number becomes 6, condition is False -> loop
  stops

```

Output

1
2
3
4
5

✗ Infinite Loop

If the loop variable is never updated, the condition stays True forever. Press Ctrl+C to stop.

```

1 number = 1
2 while number <= 5:
3     print(number) # INFINITE: number never changes!

```

Math Bridge: Sum of First n Natural Numbers

$$S = 1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$$

Use the formula for speed; use a loop to understand accumulation.

Program: Sum of Natural Numbers (Two Methods)

```

1 n = int(input("Enter n: "))
2
3 # METHOD 1: Direct formula O(1) time
4 formula_sum = n * (n + 1) // 2
5 print("Formula sum:", formula_sum)
6
7 # METHOD 2: while loop accumulator O(n) time
8 loop_sum = 0 # accumulator starts at 0
9 i = 1
10 while i <= n:
11     loop_sum += i # add i to running total
12     i += 1 # advance counter
13 print("Loop sum: ", loop_sum)

```

Output

Enter n: 10
Formula sum: 55
Loop sum: 55

Math Bridge: GCD (Euclid's Algorithm)

$\gcd(a, b) = \gcd(b, a \bmod b)$, with $\gcd(a, 0) = a$.
Invariant: $\gcd(a, b)$ is unchanged at every step.

Program: GCD by Euclid's Algorithm

```

1 a = int(input("Enter a: "))
2 b = int(input("Enter b: "))
3 p, q = a, b # save originals for display
4
5 while b != 0:
6     a, b = b, a % b # Euclid step: gcd(a,b)=gcd(b,
7     a%b)
8 print(f"gcd({p},{q}) = {a}")
9 # e.g. gcd(48,36) = 12

```

1.26.10 The for Loop

The `for` loop iterates over every item in a sequence.

> Syntax: for Loop

```

for variable in sequence:
    statements

```

Program: for Loop Examples

```

1 # Over a list of subjects
2 subjects = ["Maths", "Physics", "Computer Science"]
3 for subject in subjects:
4     print(subject) # printed one per iteration
5
6 # Over a string (character by character)
7 for ch in "PYTHON":
8     print(ch, end=" ") # P Y T H O N
9
10 # Over dictionary (key-value pairs)
11 student = {"name": "Anu", "age": 17}
12 for key, val in student.items():
13     print(f" {key}: {val}")

```

Output

Maths
Physics
Computer Science
P Y T H O N
name: Anu
age: 17

1.26.11 The range() Function

The `range()` function generates a lazy sequence of integers.

► Syntax: range()

```
range(stop)           # 0, 1, ..., stop-1
range(start, stop)     # start, ..., stop-1
range(start, stop, step) # custom stride
```

► Program: range() Variants

```
1 print(list(range(5)))           # [0, 1, 2, 3, 4]
2 print(list(range(2, 8)))        # [2, 3, 4, 5, 6, 7]
3 print(list(range(2, 11, 2)))     # [2, 4, 6, 8, 10]
4 print(list(range(5, 0, -1)))     # [5, 4, 3, 2, 1]
5
6 # Print numbers 1 to 5
7 for number in range(1, 6):      # stop=6 is NOT
    included
8     print(number, end=" ")      # 1 2 3 4 5
```

Output

```
[0, 1, 2, 3, 4]
[2, 3, 4, 5, 6, 7]
[2, 4, 6, 8, 10]
[5, 4, 3, 2, 1]
1 2 3 4 5
```

1.26.12 Loop with Accumulator

An accumulator collects a running result (sum, product, count).

► Program: Sum Accumulator

```
1 total = 0                     # accumulator starts at 0
2
3 for number in range(1, 6):
4     total += number           # add each number to total
5
6 print("Sum =", total)        # 15
```

Output

```
Sum = 15
```

Trace table:

number	total after addition
1	1
2	3
3	6
4	10
5	15

◆ Math Bridge: Factorial

$n! = 1 \times 2 \times \dots \times n$, $0! = 1$.

Example: $5! = 120$. Use a **product accumulator** (starts at 1).

► Program: Factorial Using a Loop

```
1 n = int(input("Enter n: "))
2 fact = 1           # product accumulator starts at 1
3
4 for i in range(1, n + 1):
5     fact = fact * i # multiply each term
6
7 print(f"{n}! = {fact}")
8 # e.g. 5! = 120
```

◆ Math Bridge: Fibonacci Sequence

$F_0 = 0$, $F_1 = 1$, $F_n = F_{n-1} + F_{n-2}$.

Sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

► Program: Fibonacci Sequence

```
1 n = int(input("How many terms? "))
2 a, b = 0, 1           # first two Fibonacci terms
3
4 for i in range(n):
5     print(a, end=" ") # print current term
6     a, b = b, a + b   # advance: next = sum of
    previous two
7
8 # For n=8: 0 1 1 2 3 5 8 13
```

1.26.13 The break Statement

The break statement immediately terminates the nearest enclosing loop.

► Syntax: break

```
break
```

► Program: Linear Search Using break

```
1 numbers = [10, 25, 40, 55, 70]
2 target = 40
3
4 for number in numbers:
5     if number == target:
6         print("Target found")
7         break           # stop searching
    immediately
8
9 print("Search completed") # always executes
```

Output

```
Target found
Search completed
```

1.26.14 The continue Statement

The continue statement skips the remaining body of the current iteration and moves to the next iteration.

► Syntax: continue

```
continue
```

► Program: Print Only Odd Numbers

```
1 for number in range(1, 11):
2     if number % 2 == 0:
3         continue       # even: skip to next iteration
4     print(number)       # only odd numbers reach here
```

Output

```
1 3 5 7 9
```

1.26.15 The pass Statement

The pass statement does nothing. It is a placeholder.

► Syntax: pass

```
pass
```

► Program: pass as Placeholder

```
1 mark = 88
2
3 if mark >= 90:
4     pass               # to be implemented later
5 else:
6     print("Need a little more effort") # executes
```

Output

Need a little more effort

Use of pass

The pass statement is useful while designing a program step by step. It allows an empty block temporarily without causing a syntax error.

1.26.16 Loop else Clause

The else block executes when the loop finishes *normally* (without break).

Syntax: Loop else

```
for variable in sequence:
    statements
else:
    statements_after_normal_completion
```

Program: Search Using Loop else

```
1 numbers = [10, 20, 30, 40]
2 target = 25
3
4 for number in numbers:
5     if number == target:
6         print("Target found")
7         break
8         # loop did NOT complete normally
9 else:
10    # This runs ONLY if break was never executed
11    print("Target not found")
```

Output

Target **not** found

Math Bridge: Prime Numbers

$n > 1$ is prime if it has no divisor d with $2 \leq d \leq \sqrt{n}$. Why check only to $\sqrt{n}$? If $n = a \times b$ and both $> \sqrt{n}$, then $ab > n$ – impossible. So one factor must be $\leq \sqrt{n}$.

Example 1: Testing if 17 is Prime

$\sqrt{17} \approx 4.12$
 Check divisors d where $2 \leq d \leq 4$:
 $17 \div 2 = 8.5 \rightarrow$ Not divisible
 $17 \div 3 = 5.67 \rightarrow$ Not divisible
 $17 \div 4 = 4.25 \rightarrow$ Not divisible
 No divisors found up to $\sqrt{17}$.
 $\therefore$ 17 is a prime number.

Example 2: Testing if 36 is Composite

$\sqrt{36} = 6$
 Check divisors d where $2 \leq d \leq 6$:
 $36 \div 2 = 18 \rightarrow$ Divisible! Found divisor $d = 2$
 $36 \div 3 = 12 \rightarrow$ Divisible!
 $36 \div 4 = 9 \rightarrow$ Divisible!
 $36 \div 6 = 6 \rightarrow$ Divisible!
 Found divisor $d = 2 \leq \sqrt{36}$
 $\therefore$ 36 is NOT a prime number.

Program: Prime Checker Using Loop else

```
1 n = int(input("Enter a number: "))
2
3 if n <= 1:
4     print("Not prime")
5 else:
6     # Check divisors from 2 to floor(sqrt(n))
7     for i in range(2, int(n**0.5) + 1):
8         if n % i == 0:
9             print(f"{n} is NOT prime (divisible by {i})")
10            break
11 else:
12    # else executes only if no break occurred
13    print(f"{n} is PRIME")
```

1.26.17 Nested Loops

A nested loop is a loop inside another loop.

Syntax: Nested Loop

```
for outer in outer_sequence:
    for inner in inner_sequence:
        statements
```

Program: Multiplication Table (Nested Loops)

```
1 for i in range(1, 4):           # outer: row
2     for j in range(1, 4):       # inner: column
3         print(i * j, end=" ")  # product
4     print()                     # newline after each row
```

Output

```
1 2 3
2 4 6
3 6 9
```

Math Bridge: Armstrong Number

A k -digit number n is Armstrong if $n = d_1^k + d_2^k + \dots + d_k^k$.
 Example: $153 = 1^3 + 5^3 + 3^3$.

Program: Armstrong Number Checker

```
1 n = int(input("Enter a number: "))
2 digits = list(map(int, str(n))) # split into digit list
3 k = len(digits)                 # number of digits
4 total = sum(d ** k for d in digits) # sum each digit^k
5
6 if total == n:
7     print("Armstrong number")
8 else:
9     print("Not Armstrong")
```

Math Bridge: Perfect Number

n is **perfect** if the sum of its proper divisors equals n .
 Example: $6 = 1 + 2 + 3$.

Program: Perfect Number Checker

```
1 n = int(input("Enter a number: "))
2 total = 0
3
4 for i in range(1, n):           # proper divisors: 1 to n-1
5     if n % i == 0:               # i divides n exactly
6         total += i
7
8 if total == n:
9     print("Perfect number")
10 else:
```